

Document number: P1772R1
Revises: P1772R0
Date: 2019-10-07
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: EWGI, SG7
Reply to: Vincent Reverdy
University of Illinois at Urbana-Champaign
vince.rev@gmail.com

Variadic overload sets and overload sequences

Abstract

This document constitutes a short remark to [P1170: Overload sets as function parameters](#) and [P0051: C++ generic overload function](#). From a library design standpoint we argue that the functionality provided by these papers should be considered together, and not independently. In particular, we feel that the current form of P1170 will not solve the problem of variadic overload sets built from non-inheritable classes that P0051 is facing. Making `std::overload_set` a variadic template class would allow to solve this problem.

Contents

1	History	2
2	Background	2
2.1	Overload sets as function parameters	2
2.2	Generic overload function	3
3	The problem	4
4	Suggested resolution	6
5	References	7

1 History

- **P1772R0/D1772R0**

- Reviewed by LEWGI in Cologne
- Positive feedback for the general suggested direction
- Poll: “We want a variadic `std::overload_set` library API (as described by P1772) instead of 2 orthogonal but composable library APIs (as in both `std::overload` and `std::overload_set`, with necessary fixes to make them composable).”

Strongly For	For	Neutral	Against	Strongly Against
4	6	4	0	1

- Forwarded to EWGI

2 Background

2.1 Overload sets as function parameters

To summarize the problem that [P1170](#) is proposing to solve, we start with the following example, that currently works without problem:

```
Invoke a single function
1 double f(double x) {return x;}
2
3 auto result = std::invoke(f, 1.);
```

However, if new overloads are introduced, possibly elsewhere in the program, the call is not working anymore as in the following example:

```
Cannot invoke an overload set
1 double f(double x) {return x;}
2 double f(double x, double y) {return x * y;}
3 double f(double x, double y, double z) {return x * y * z;}
4
5 // The following is not working today
6 auto result = std::invoke(f, 1.);
```

To make it work today, the function overload set needs to be wrapped in a lambda:

```
Workaround
1 double f(double x) {return x;}
2 double f(double x, double y) {return x * y;}
3 double f(double x, double y, double z) {return x * y * z;}
4
5 auto result = std::invoke([](auto... args){return f(args...);}, 1.);
```

In fact, to achieve perfect forwarding, the lambda needs to be defined in the following much more complicated way:

```
The exact workaround
1 [&](auto&& ... args)
2     noexcept(noexcept(f(std::forward<decltype(args)>(args)...)))
3     -> decltype(f(std::forward<decltype(args)>(args)...)) {
4     return f(std::forward<decltype(args)>(args)...);
5 }
```

[P1170](#) proposes to introduce a magic class to wrap overload sets:

Hypothetical overload set wrapper proposed by P1170

```

1  template <typename T>
2  class overload_set
3  {
4      // Data members
5      private:
6      T f;
7
8      // Lifecycle
9      public:
10     overload_set(/* unspecified */);
11     overload_set(overload_set const&) = default;
12     overload_set(overload_set&&) = default;
13     overload_set& operator=(overload_set const&) = default;
14     overload_set& operator=(overload_set&&) = default;
15     ~overload_set() = default;
16
17     // Invocation
18     public:
19     template <typename... Us>
20     invoke_result_t<F&, Us...> operator()(Us&&... us) &
21         noexcept(is_nothrow_invocable_v<F&, Us...>);
22     template <typename... Us>
23     invoke_result_t<F const&, Us...> operator()(Us&&... us) const&
24         noexcept(is_nothrow_invocable_v<F const&, Us...>);
25     template <typename... Us>
26     invoke_result_t<F, Us...> operator()(Us&&... us) &&
27         noexcept(is_nothrow_invocable_v<F, Us...>);
28     template <typename... Us>
29     invoke_result_t<F const, Us...> operator()(Us&&... us) const&&
30         noexcept(is_nothrow_invocable_v<F const, Us...>);
31 };

```

`std::overload_set` would work on overload sets, but also with any callable (lambdas, function objects, function pointers, member function pointers...). In that sense, it would constitute a sort of universal wrapper for callables.

2.2 Generic overload function

P0051 proposes to introduce a `std::overload` function to make several callable members of the same overload set:

Use case of generic overload function

```

1  auto f = std::overload(
2      [](int x){return x;},           // 1
3      [](double x){return x;},       // 2
4      [](int x, int y){return x * y;}, // 3
5      [](double x, double y){return x * y;}, // 4
6      [](int x, int y, int z){return x * y * z;}, // 5
7      [](double x, double y, double z){return x * y * z;}, // 6
8  );
9
10 f(1., 2., 3.); // Will call the 6th version

```

It also proposes a `std::first_overload` that will pick the first function that can be called on the provided parameters as in:

Use case of generic first overload function

```
1 auto f = std::first_overload(  
2     [](int x){return x;},           // 1  
3     [](double x){return x;},       // 2  
4     [](int x, int y){return x * y;}, // 3  
5     [](double x, double y){return x * y;}, // 4  
6     [](int x, int y, int z){return x * y * z;}, // 5  
7     [](double x, double y, double z){return x * y * z;}, // 6  
8 );  
9  
10 f(1., 2., 3.); // Will call the 5th version
```

In terms of implementation, the way to make it work, is by inheriting from the callables as in:

Hypothetical overload set wrapper to make P0051 work

```
1 template <class... F>  
2 class overload_set: F...  
3 {  
4     public:  
5     using F::operator()...;  
6  
7     /* ... */  
8 };
```

This would only work however for inheritable classes. Functions, function pointers, member function pointers, and other non-inheritable callables can be wrapped in an inheritable class. However, there is currently no workaround for non-inheritable classes as in:

The problem of non-inheritable classes

```
1 union union_int  
2 {  
3     int operator()(int x){return x;}  
4 };  
5  
6 union union_double  
7 {  
8     double operator()(double x){return x;}  
9 };  
10  
11 union_int ui;  
12 union_double ud;  
13  
14 auto f = std::overload(ui, ud); // Will not work
```

Two possible solutions are either to make `std::overload` rely on some compiler magic, or to find hacky tricks based on future reflection facilities.

3 The problem

However the problem goes beyond non-inheritable classes. The current designs proposed by [P1170](#) and [P0051](#) seem to be incompatible with each other. The problem arises when combining the two as in the following example:

Problem when combining overload set and overload function

```
1 // Overload set with floating-points  
2 double f(double x) {return x;}  
3 double f(double x, double y) {return x * y;}
```

```

4 double f(double x, double y, double z) {return x * y * z;}
5
6 // Overload set with integers
7 int g(int x) {return x;}
8 int g(int x, int y) {return x * y;}
9 int g(int x, int y, int z) {return x * y * z;}
10
11 // Combining std::overload_set and std::overload
12 auto function = std::overload(
13     std::overload_set(f),
14     std::overload_set(g)
15 );
16
17 function(1., 2., 3.); // Does not work: ambiguous!

```

It would be natural for users to expect the following:

Expectation

```

1 struct function
2 {
3     int operator()(int x) {return x;}
4     double operator()(double x) {return x;}
5     int operator()(int x, int y) {return x * y;}
6     double operator()(double x, double y) {return x * y;}
7     int operator()(int x, int y, int z) {return x * y * z;}
8     double operator()(double x, double y, double z) {return x * y * z;}
9 };

```

but in reality it produces the following:

Reality

```

1 struct function
2 {
3     // Coming from f
4     template <typename... Us>
5     invoke_result_t<F&, Us...> operator()(Us&&... us) &
6         noexcept(is_nothrow_invocable_v<F&, Us...>);
7     template <typename... Us>
8     invoke_result_t<F const&, Us...> operator()(Us&&... us) const&
9         noexcept(is_nothrow_invocable_v<F const&, Us...>);
10    template <typename... Us>
11    invoke_result_t<F, Us...> operator()(Us&&... us) &&
12        noexcept(is_nothrow_invocable_v<F, Us...>);
13    template <typename... Us>
14    invoke_result_t<F const, Us...> operator()(Us&&... us) const&&
15        noexcept(is_nothrow_invocable_v<F const, Us...>);
16
17    // Coming from g
18    template <typename... Us>
19    invoke_result_t<F&, Us...> operator()(Us&&... us) &
20        noexcept(is_nothrow_invocable_v<F&, Us...>);
21    template <typename... Us>
22    invoke_result_t<F const&, Us...> operator()(Us&&... us) const&
23        noexcept(is_nothrow_invocable_v<F const&, Us...>);
24    template <typename... Us>
25    invoke_result_t<F, Us...> operator()(Us&&... us) &&
26        noexcept(is_nothrow_invocable_v<F, Us...>);
27    template <typename... Us>
28    invoke_result_t<F const, Us...> operator()(Us&&... us) const&&

```

```

29     noexcept(is_nothrow_invocable_v<F const, Us...>);
30 };

```

which leads to the ambiguity at the call site.

4 Suggested resolution

Given these incompatibilities, we suggest that the handling of overload sets should be unified instead of separated in two proposals:

Suggested hypothetical resolution

```

1  template <class... F>
2  class overload_set: overload_set<F>...
3  {
4      public:
5      using overload_set<F>::operator()...;
6
7      /* ... */
8
9  };
10
11 template <class F>
12 class overload_set<F>
13 {
14     // Data members
15     private:
16     T f;
17
18     // Lifecycle
19     public:
20     overload_set(/* unspecified */);
21     overload_set(overload_set const&) = default;
22     overload_set(overload_set&&) = default;
23     overload_set& operator=(overload_set const&) = default;
24     overload_set& operator=(overload_set&&) = default;
25     ~overload_set() = default;
26
27     // Invocation
28     public:
29     /* ... compiler magic to import the overload set as is ... */
30     /* ... replacing the name of the functions by operator() ... */
31 };

```

so that the following works as expected:

Example of use of the resolution

```

1  // Overload set with floating-points
2  double f(double x) {return x;}
3  double f(double x, double y) {return x * y;}
4  double f(double x, double y, double z) {return x * y * z;}
5
6  // Overload set with integers
7  int g(int x) {return x;}
8  int g(int x, int y) {return x * y;}
9  int g(int x, int y, int z) {return x * y * z;}
10
11 // Variadic overload set
12 auto function = std::overload_set(f, g);

```

```

13
14 // Calls "double f(double x, double y, double z) {return x * y * z;}"
15 function(1., 2., 3.);

```

Based on the same principle, a whole family of utilities can be created:

```

                                Overload set utilities
1 // Picks the best overload according to overload resolution
2 template <class... F>
3 class overload_set;
4
5 // Picks the first overload that works
6 template <class... F>
7 class overload_sequence;
8
9 // Overload set with specific result (like is_invocable_r)
10 template <class R, class... F>
11 class overload_set_r;
12
13 // Overload sequence with specific result (like is_invocable_r)
14 template <class R, class... F>
15 class overload_sequence_r;

```

At this point, this proposal suggests a direction of investigation, but does not provide wording. The main goal was to raise the issue about the incompatibility of approaches in existing proposals [P1170: Overload sets as function parameters](#) and [P0051: C++ generic overload function](#) and start a discussion on how to properly solve the problem. The authors think that the suggested approach would work at the price of some compiler magic (that [P1170](#) requires anyway) but other suggestions are very welcomed. What is certain, however, is that [P1170](#) and [P0051](#) propose partial solutions of a more general problem, and these solutions are currently not compatible with each other.

5 References

- [P1170: Overload sets as function parameters](#), Barry Revzin and Andrew Sutton, *ISO / IEC – JTC1 / SC22 / WG21* (October 2018)
- [P0051: C++ generic overload function](#), Vicente J. Botet Escriba, *ISO / IEC – JTC1 / SC22 / WG21* (November 2015)
- [Custom Overload Sets and Inline SFINAE for Truly Generic Interfaces](#), Vincent Reverdy, *Cp-pCon 2018* (September 2018)